Professional Training in Cybersecurity I

Module 1 — Session 1

Encapsulation, Forwarding and the Byte-Level Frame

Mohamed Gebril

Session Objective

By the end you can do each of these, on the reference frame's real bytes:

Trace — one message down the stack and back up
the headers the sender adds, and the checks the receiver runs

Follow — a packet across a network
what each switch, router and Internet edge rewrites, and what survives

Read — a header straight from raw bytes
offsets, hex and nibbles, network byte order, next-layer pointers

Recognize — four header layouts
IPv4 and TCP field by field in the RFC drawing, with IPv6 and UDP for contrast

Build — the reference frame by hand, and parse it back
`struct` pack and unpack, bit fields, the payload boundary, the checksum

Transfer — the method to a protocol you have not met
an unknown header is a new layout, read with the same skill

Session Agenda

Part 1 — Layering and Encapsulation
three addresses, wrapping at the sender, the receiver's checks, how deep each device reads

Part 2 — Forwarding Across a Network
what each hop rewrites, inside a router, the Internet edge, the MTU and the MSS

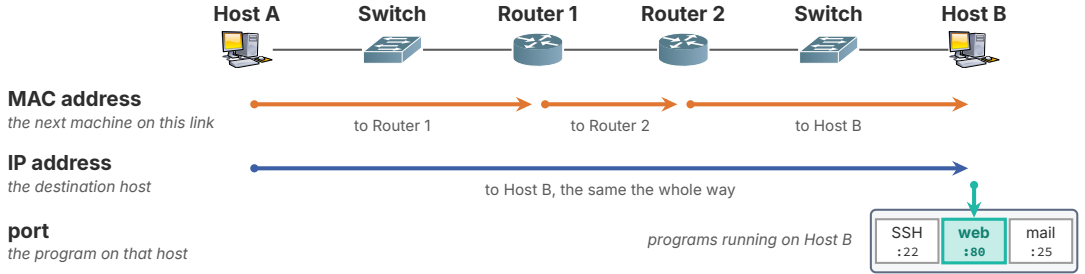
Part 3 — The Frame as a Byte Sequence
the 59-byte reference frame, hex and nibbles, byte order, `struct` format strings

Part 4 — Protocol Header Structure
the RFC header diagram, IPv4 and TCP, with IPv6 and UDP for contrast

Part 5 — Frame Construction and Parsing
building in code, bit fields, parsing, the payload boundary and padding, the checksum

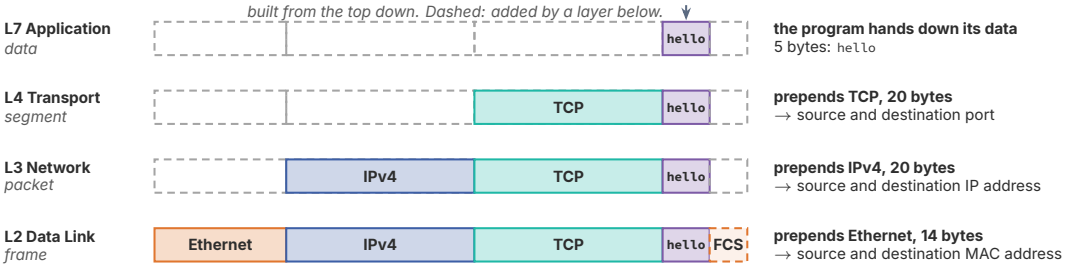
Layering and Encapsulation

The Problem: One Message Needs Three Addresses



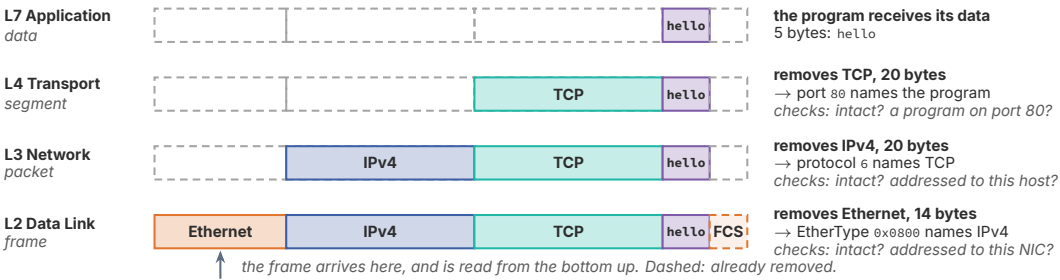
- Delivering one message from a program on Host A to a program on Host B takes **three addresses**. Each names a different thing, and no single address can name all three.
- A **MAC address** is the 48-bit address of one network interface card (**NIC**). It is valid only on one **link**, the network that switches connect directly, so this path needs three, one per link.
- An **IP address** is the 32-bit address of one host, valid across every link on the path. A **port** is a 16-bit number that names one program on that host.
- The message must carry all three addresses at once, one inside the next. That nesting is **encapsulation**.

Encapsulation: Each Layer Prepends Its Header



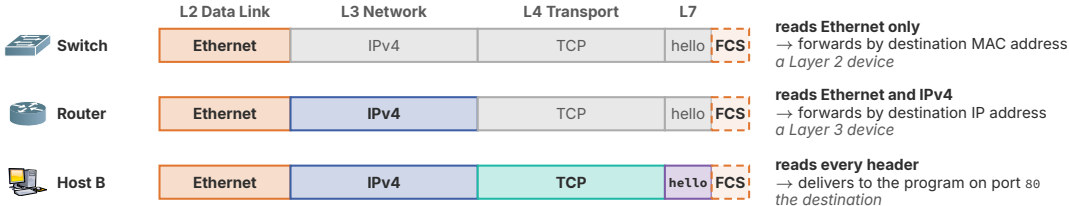
- A **layer** is one level of the network stack with one job, and a **header** is the block of fields that layer places in front of the data to do that job.
- Encapsulation runs top-down: from L7, each layer prepends its header to what the layer above handed it, and the result takes that layer's name: a **segment** at L4, a **packet** at L3, a **frame** at L2. The NIC then appends the **Frame Check Sequence (FCS)**, a 4-byte CRC-32 error check over the frame.
- The layer numbers come from the seven-layer **OSI model**, but this frame carries only four. L1 is the signal on the wire, not bytes, and the application itself does the jobs of L5 and L6, so they add no header.

Decapsulation: Each Layer Checks, Strips, Passes Up



- **Decapsulation** is encapsulation in reverse: from L2 upward, each layer checks the unit it receives (frame, packet, segment), removes its own header, and passes the rest up.
- Each check asks two things: *is this addressed to me, and did it arrive intact?* If both pass, a field in the header, such as the EtherType, names the protocol that receives the rest.
- A failed check is a **silent drop**: the layer discards the unit, and the sender sees only that no reply came. If the only problem is that no program listens on the port, TCP answers with a reset (**RST**) instead.

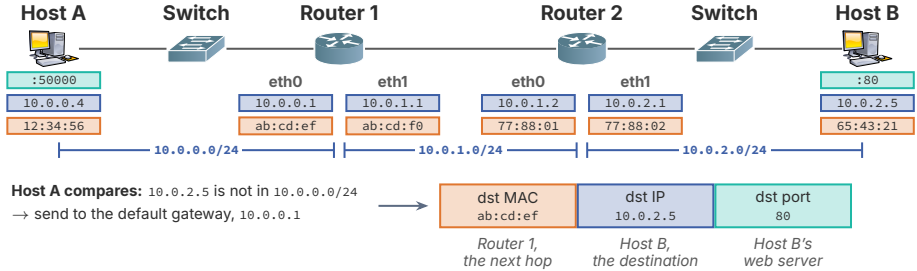
Layer 2 and Layer 3 Devices: How Deep Each One Reads



- A **Layer 2 device**, such as a switch, reads only the Ethernet header and forwards by MAC address. A **Layer 3 device**, such as a router, also reads the IP header and forwards by IP address.
- Each device reads only as deep as its job needs and forwards the rest unread. Only the destination host reads every header.
- Reading less is not protection: any device on the path **can read every byte**, plaintext included.
- A NIC in **promiscuous mode** skips the destination MAC check and accepts every frame on the link. Traffic capture, later in the module, depends on it.

Forwarding Across a Network

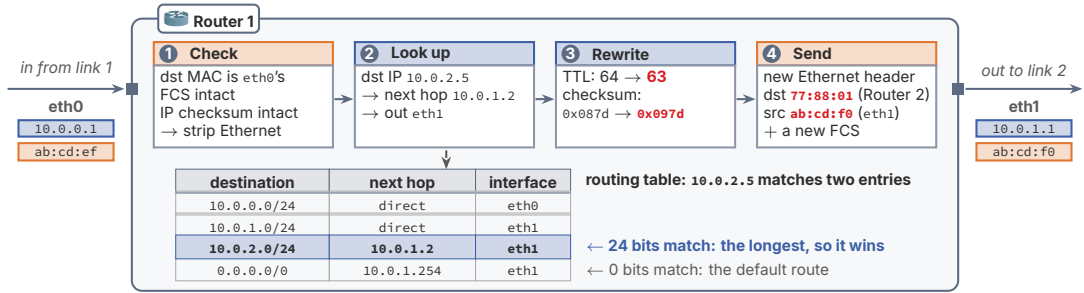
Forwarding: Host A Sends to Its Gateway, Not to Host B



MACs show their last three bytes. All begin `08:00:27`.

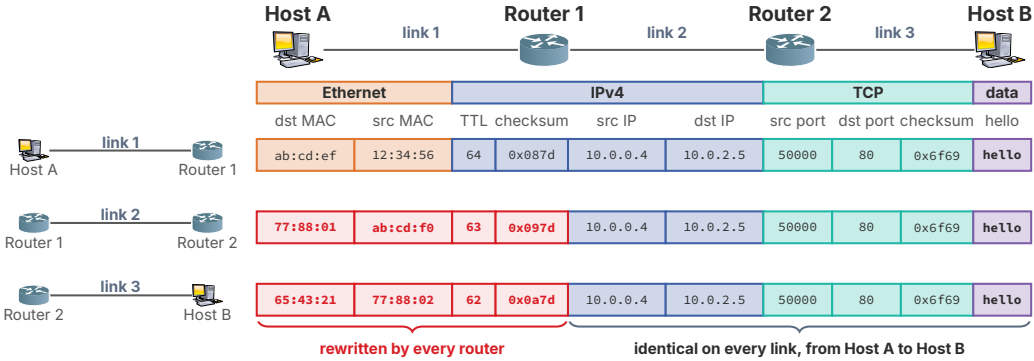
- **Forwarding** moves a frame toward its destination one link at a time: a router receives it on one link and sends it out on the next. Each link crossed is one **hop**.
- A **subnet** is the block of IP addresses assigned to one link, written as a prefix: `10.0.0.0/24` fixes the first 24 bits. A host delivers directly only inside its own subnet.
- For any other destination, a host sends the frame to its **default gateway**, the router on its own link. The destination MAC names Router 1, while the destination IP still names Host B.

Inside a Router: Check, Look Up, Rewrite, Send



- A **routing table** lists the networks a router can reach. Each entry gives the **next hop**, the router to hand the packet to, and the interface to send it from. *direct* means the network is on that link.
- When several entries match, the router uses the most specific one: the **longest prefix match**. The **default route**, `0.0.0.0/0`, matches every address, so it catches what no longer prefix does.
- The **TTL** (time to live) drops by one at every router, and a packet that reaches 0 is discarded, so a routing loop cannot carry it forever. Router 2's MAC comes from **ARP**, which finds the MAC for an IP address on one link, taught later in the module.

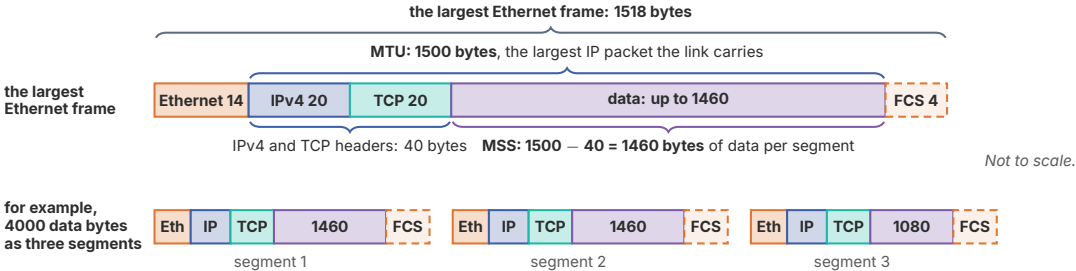
Hop by Hop: What Changes and What Survives



Red: rewritten by the router before that link. MACs show their last three bytes. All begin `08:00:27`.

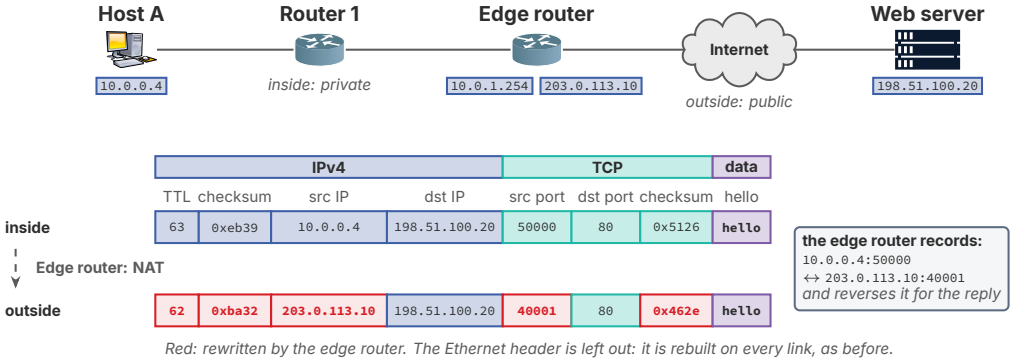
- Each router builds a new Ethernet header, so every link carries its own MAC pair.
- Only the TTL and the IP checksum change. Addresses, ports, TCP checksum and data arrive as sent.
- The TCP checksum covers the IP addresses but not the TTL, so it survives. Each NIC appends a new FCS.

The MTU: How Much One Link Carries



- The **MTU** (maximum transmission unit) is the largest IP packet one link carries: 1500 bytes on Ethernet. With its header and FCS, the largest Ethernet frame is 1518 bytes.
- IPv4 and TCP take 40 of those bytes, which leaves 1460 for data: the **MSS** (maximum segment size). TCP sends larger data as several segments of at most 1460 bytes.
- The reference frame sets **Don't Fragment (DF)**: a router whose next link has a smaller MTU drops it and sends back an ICMP error, instead of splitting it into fragments.

The Internet Edge: NAT Rewrites the Source

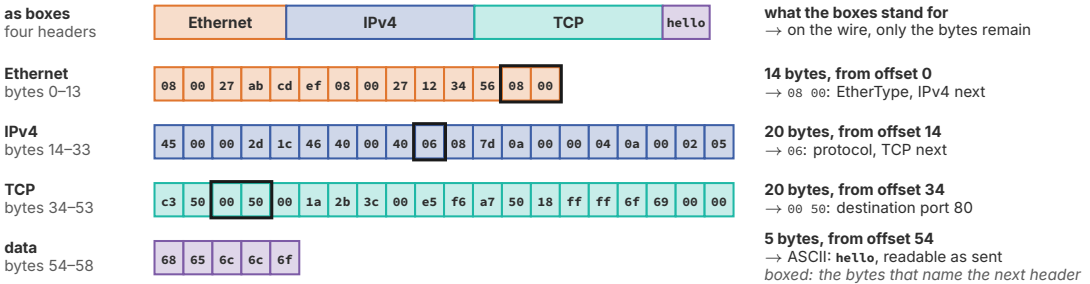


Red: rewritten by the edge router. The Ethernet header is left out: it is rebuilt on every link, as before.

- A **private address**, such as any in `10.0.0.0/8`, is valid only inside its own network, and Internet routers carry no route to it. Every address on this campus is private.
- At the edge, **NAT** (network address translation) swaps the private source for the router's public address and a port it picks, then recomputes both checksums. Many routers keep the original port when it is free.

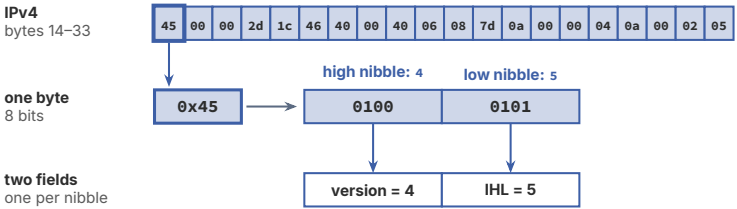
The Frame as a Byte Sequence

The Reference Frame: 59 Bytes at Fixed Offsets



- A **byte** is 8 bits, and a frame is a run of bytes. A field's position is its **offset**, counted from 0: Ethernet fills bytes 0 to 13, IPv4 14 to 33, TCP 34 to 53, and the data 54 to 58.
- Nothing in the bytes marks where one header ends. Sender and receiver agree on every field's offset and width, and a reader that starts one byte off gets every value wrong.
- A **hexdump** shows each byte as two hexadecimal (base-16) digits. Read as ASCII text, the data bytes spell `hello`, exactly as sent. These are the 59 bytes Host A's software hands to its NIC.

Reading Bytes: Hex, Nibbles, and Bits



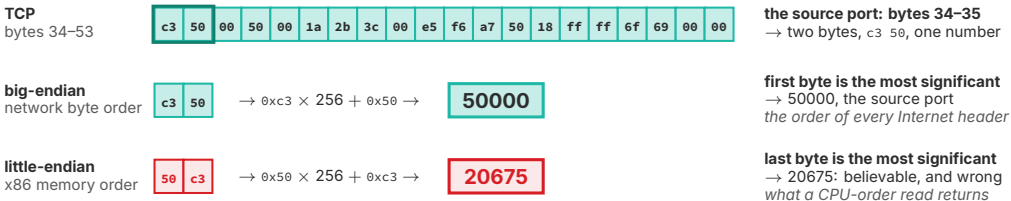
byte 14: the first IPv4 byte
→ `0x45`, two hex digits

two nibbles of 4 bits
→ hex `4` = `0100`, hex `5` = `0101`
each hex digit is exactly 4 bits

version 4: this is IPv4
→ IHL 5 × 4 bytes = a 20-byte header
code splits them later in this session

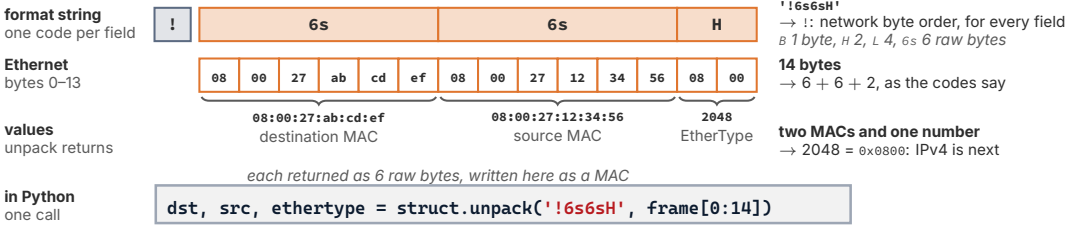
- A **bit** is a single 0 or 1, and a **nibble** is 4 bits. **Hexadecimal** writes each nibble as one digit from 0 to f, so a byte is always two hex digits: `0x45` is `0100 0101`, not forty-five.
- IPv4 packs two fields into byte 14: the high nibble is the version, 4, and the low nibble is the **IHL** (Internet Header Length), 5.
- The IHL counts 4-byte words, so 5 means a 20-byte header. Header fields narrower than a byte are common, and code splits them with bit operations, later in this session.

Byte Order: The Endianness Trap



- A field wider than one byte must be read in an agreed order. In **big-endian** order the first byte is the most significant, so `c3 50` is `0xc350`, 50000: the source port.
- Internet headers are all big-endian, so this order is also called **network byte order**. x86 processors store numbers **little-endian**, with the last byte most significant.
- Read the port in the processor's order and you get `0x50c3`, 20675: a believable number, and wrong. Every multi-byte field has to be read in network byte order.

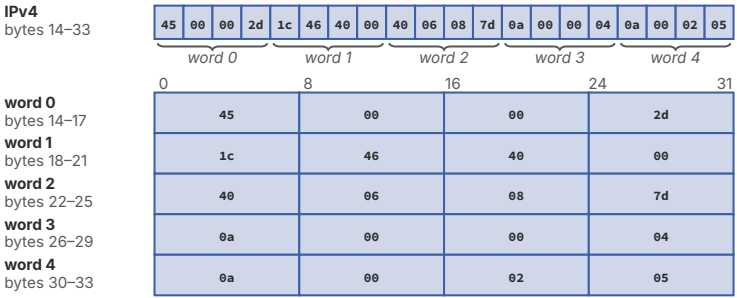
Reading a `struct` Format String



- A **format string** tells Python's `struct` module how to read bytes: one code per field, left to right, each a fixed width. `unpack` turns bytes into values, and `pack` turns values back into bytes.
- `'!6s6sH'` reads an Ethernet header: `!` sets network byte order, each `6s` takes 6 raw bytes (a MAC), and `H` reads 2 bytes as one number, the EtherType.
- A parser trusts the offsets and lengths it reads. A field that lies, such as a length larger than the data behind it, makes a careless parser read past the end: malformed input like this is behind many parser bugs met later in the module.

Protocol Header Structure

Header Diagrams: Fields in Rows of 32 Bits

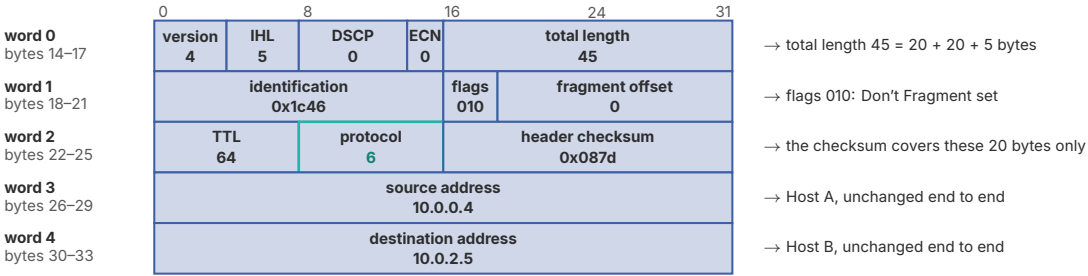


one row of 20 bytes
→ in the order they are sent

the RFC drawing: 32 bits per row
→ read left to right, then down
→ 5 rows: IHL 5, a 20-byte header

- An **RFC** (Request for Comments) is one of the numbered documents that define the Internet's protocols. Each draws its header in rows of 32 bits, as here.
- One row is a **word**: 32 bits, 4 bytes. The scale on top numbers each **bit position**, 0 to 31, so a field's width is its share of the row.
- The rows read left to right, then down, which is the order the bytes are sent in. IPv4's IHL counts these rows: 5 words, 20 bytes.

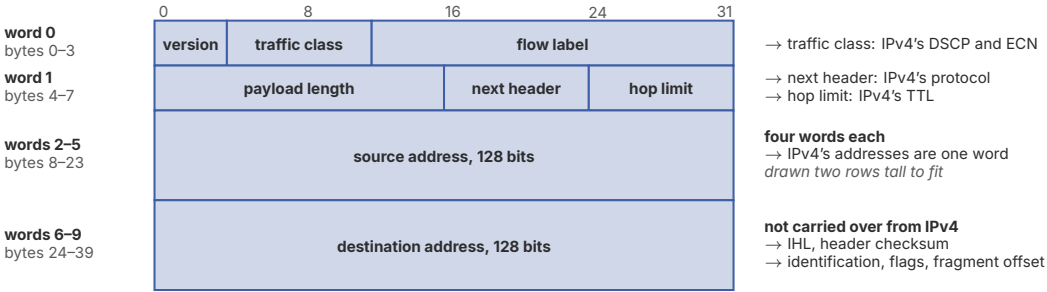
IPv4: 20 Bytes, Two Lengths, Two Addresses



these five words are `frame[14:34]`, in the RFC 791 layout

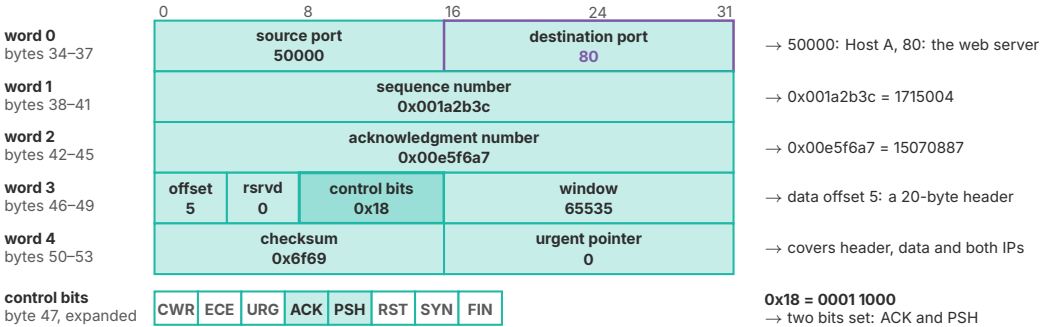
- **Total length** counts the whole packet, header and data, while the IHL counts the header alone.
- The header grows past 20 bytes only when it carries **options**, extra fields after the addresses, up to 60 bytes (IHL 15). This one carries none.
- Identification, flags and fragment offset let a receiver rebuild a packet that was split on the way. DSCP marks a priority class, and ECN signals congestion. Both are covered later in the module.

IPv6: A Fixed 40-Byte Header With 128-Bit Addresses



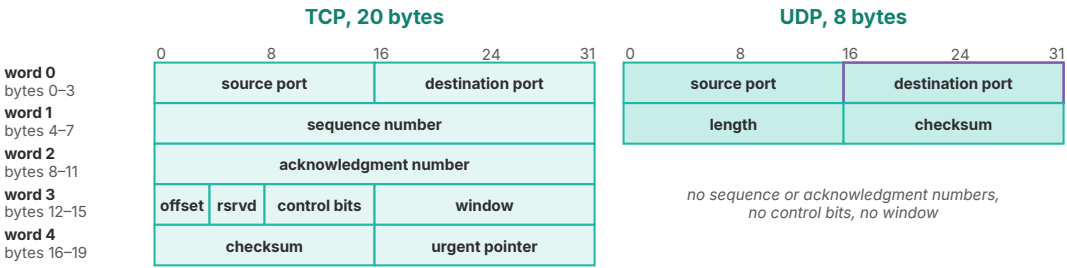
- **IPv6** is the newer version of IP, with 128-bit addresses. Its header is a fixed 40 bytes, 10 words, and the two addresses fill 8 of them.
- It has no header checksum, because the link and transport checksums already cover the packet. It has no fragmentation fields either: only the sender may split an IPv6 packet.
- Instead of options, **next header** names what follows, an extension header or TCP, with the numbers IPv4's protocol field uses.

TCP: 20 Bytes, Ports, Sequence Numbers, Control Bits



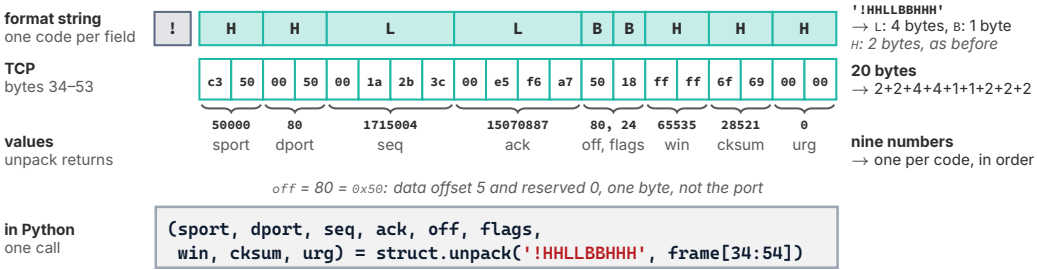
- The **sequence number** numbers the first data byte in this segment, and the **acknowledgment number** is the next byte the sender expects back. Together they let TCP find lost data and send it again.
- The **control bits**, also called flags, are 8 one-bit signals. `0x18` sets two: ACK (the acknowledgment number is valid) and PSH (deliver the data now).
- The data offset is TCP's IHL: the header's length in words, 5 here, up to 15 with options.

UDP: An 8-Byte Header With No Connection State



- A **datagram** is UDP's unit, as a segment is TCP's. Its header is four 16-bit fields, 8 bytes in all, against TCP's 20.
- **Length** counts the header and the data, so it is at least 8. A checksum of zero means the sender computed none, which IPv4 allows.
- With no sequence numbers, acknowledgments or control bits, UDP keeps no connection: a lost datagram stays lost unless the application notices.

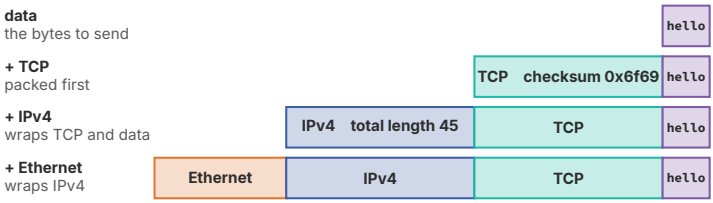
Reading TCP's Header With `'!HHLLBBHHH'`



- `'!HHLLBBHHH'` reads TCP's 20 bytes as nine fields: `H` for each 2-byte field, `L` for the 4-byte sequence and acknowledgment numbers, and `B` for a single byte.
- `struct` reads whole bytes, so byte 46 comes back as 80, `0x50`: the data offset and the reserved bits together. It is not the destination port, which is also 80.
- Splitting that byte takes bit operations, later in this session, where the same format string also builds the header.

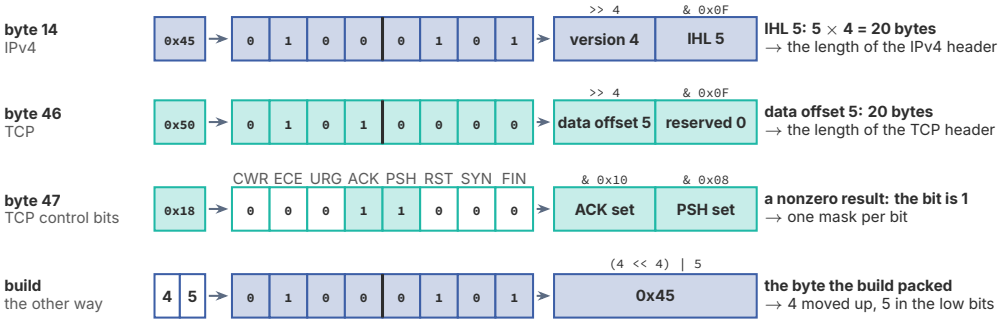
Frame Construction and Parsing

Building the Frame: Pack Each Header, Inside Out



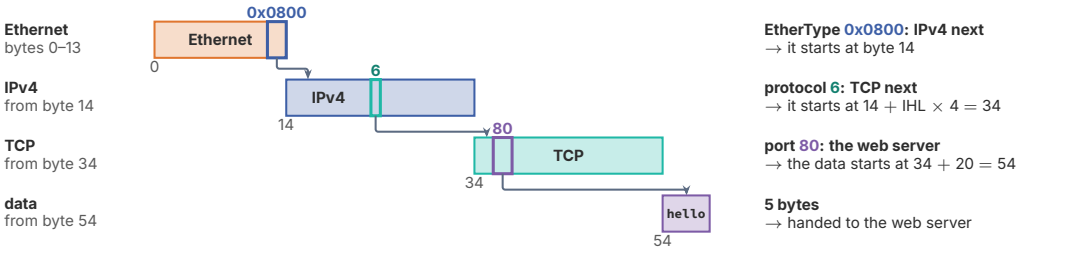
- **Building** a frame means writing each header's field values into bytes with `pack`, then joining the headers in wire order: Ethernet, IPv4, TCP, data.
- The build runs **inside out**, data first, because an outer header holds values computed from what it wraps: IPv4's total length is its own 20 bytes plus the 25 inside it.
- IPv4's format string uses only codes already met, with `4s` for each 4-byte address. The joined 59 bytes equal the reference frame exactly.

Bit Fields: Shift and Mask Split One Byte



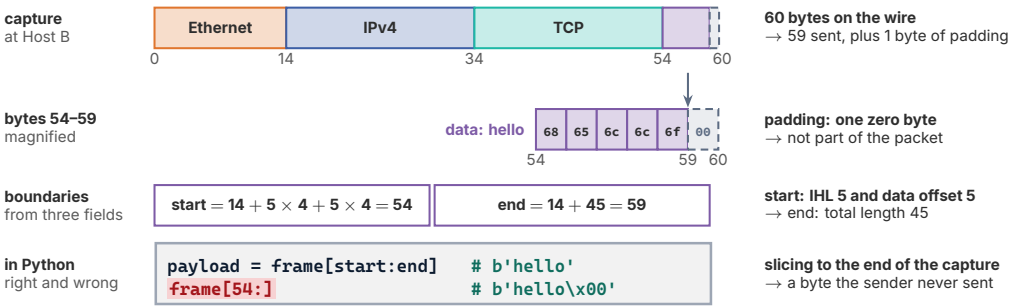
- A **bit field** is a field narrower than a byte. `frame[14]` returns the whole byte as one number, `0x45`, so the fields inside it are split with bit operations.
- A **right shift**, `>> 4`, moves the high nibble down and drops the low one. A **mask**, `& 0x0F`, is a bitwise AND that keeps only the bits set in the mask.
- A left shift, `<<`, and a bitwise OR, `|`, put two fields back into one byte.

Parsing: Unpack Each Header, Follow Its Pointer



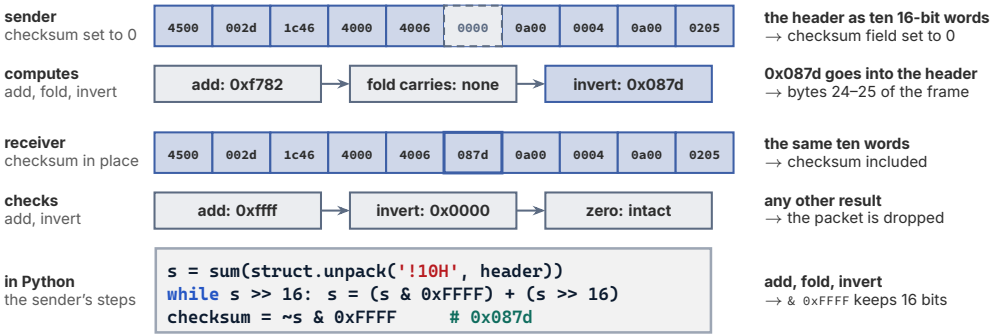
- **Parsing** is the build reversed, **outside in**: unpack one header, read the field that names the next one, then work out where the next one starts.
- TCP starts at `14 + IHL × 4`, never at a fixed 34. An IPv4 header with options is longer than 20 bytes, and its IHL says by how much.
- An unrecognized pointer, such as an EtherType other than `0x0800`, means a different header follows, so the parser stops rather than read it as IPv4.

The Payload Boundary: Computed From Header Fields



- **Padding** is filler added to reach a minimum size. An Ethernet frame carries at least 46 bytes after its header, so the 45-byte IPv4 packet gets one zero byte, and Host B captures 60 bytes.
- Padding is not part of the packet, and the total length does not count it. The data ends at `14 + total`, never at the end of the capture.
- The start comes from both header lengths, so options in either header move it.

The IPv4 Checksum: A One's-Complement Sum



- A **one's-complement sum** adds 16-bit words and adds any carry past 16 bits back in. The sender stores the sum inverted, every bit flipped by `~`, and `& 0xFFFF` keeps Python's result to 16 bits.
- The checksum catches accidental damage, not tampering. Anyone who edits a field recomputes it with these three lines, so a valid checksum proves nothing about who wrote the header, as forged packets later in the module show.

Key Takeaways

- A message carries **three addresses**: a MAC for the next machine on this link, an IP address for the host across networks, a port for the program. Each layer prepends its own header, and the receiver checks them in reverse.
- **Routers rebuild the frame on every link**: new MACs, the TTL one lower, a new IP checksum. The IP addresses and ports ride through, unless a NAT at the edge rewrites the source.
- A frame is **bytes at fixed offsets**. A field is only an agreed offset and width, and every multi-byte field is read in network byte order.
- **Read every boundary from the headers** (IHL, data offset, total length), never from a fixed number or from the size of the capture.
- **A checksum detects accidents, not tampering.** A protocol you have never met is a new layout, read with the same skill.

How the Practical Guide Uses This

1. **Open the Practical Guide.** Build the reference frame inside out and parse it back outside in, until your bytes match it exactly. The capstone hands you a header you have never seen.
2. **Record as you go.** Code, outputs and screenshots go into the **Submission worksheet** while you work, not at the end.
3. **Then the Home Task.** Apply the same method on your own. It has its own worksheet.

Keep the **Tooling Reference** (format strings, nibble math, the checksum fold) and the **Troubleshooting FAQ** beside you throughout.

Open the Practical Guide

`Module1_Session1_Practical_Guide.md`

Module 1 — Session 1
Encapsulation, Forwarding and the Byte-Level Frame